Univariate Time-Series Forecasting of Daily Maximum Temperatures in Seattle Using Stacked Long Short-Term Memory (LSTM) Networks

Viraj Jamdhade
Department of Computer Science and Engineering
Pimpri Chinchwad University, Pune
Email: virajjamdhade6@gmail.com

***Abstract*—Accurate prediction of meteorological parameters is a fundamental challenge in climate analysis and environmental data science. Traditional numerical weather prediction models rely on complex physical simulations that are computationally expensive and sensitive to initial conditions. This paper presents a univariate time-series forecasting approach using a stacked Long Short-Term Memory (LSTM) network to predict daily maximum temperatures in Seattle. The model utilizes a sliding window of ten days to forecast the subsequent day's temperature. Experimental results demonstrate that the proposed architecture effectively captures both short-term fluctuations and long-term seasonal trends.**

***Index Terms*—Time-series forecasting, LSTM, Deep Learning, Weather Prediction, RNN**

I. Introduction

Weather forecasting is a fundamental problem in environmental science and meteorology due to its widespread impact on agriculture, disaster management, transportation, energy systems, and climate monitoring. Accurate prediction of meteorological variables such as temperature, precipitation, and wind patterns plays a crucial role in decision-making processes at both local and global scales. However, weather systems are inherently complex, highly dynamic, and non-linear in nature, making precise forecasting a persistent scientific challenge. Even minor variations in initial atmospheric conditions can lead to significant deviations in predicted outcomes, a phenomenon commonly referred to as chaos in weather systems.

Traditional numerical weather prediction (NWP) models attempt to address this challenge by simulating physical processes using mathematical equations derived from fluid dynamics and thermodynamics. Although these physics-based models have been widely adopted, they are computationally expensive and require substantial computational infrastructure to operate effectively. Moreover, their accuracy heavily depends on the quality and resolution of the initial input data, which is not always available, especially for localized forecasting tasks. As a result, alternative approaches that can complement or replace conventional NWP methods have gained increasing attention in recent years.

The rapid growth of data availability and advancements in computational power have enabled the development of data-driven forecasting techniques based on machine learning and deep learning. These approaches focus on learning patterns directly from historical observations rather than explicitly modeling physical processes. While classical machine learning algorithms have shown some success, they often struggle to model temporal dependencies present in time-series data. Standard feed-forward neural networks, for example, treat input observations as independent entities and therefore fail to capture the sequential relationships that are critical in weather forecasting.

Recurrent Neural Networks (RNNs) were introduced to address this limitation by incorporating feedback connections that allow information from previous time steps to influence current predictions. However, traditional RNN architectures suffer from the vanishing gradient problem, which restricts their ability to learn long-term dependencies in sequential data. This limitation significantly affects their performance when modeling climatic and meteorological time series, where long-range temporal patterns and seasonal trends are essential.

Long Short-Term Memory (LSTM) networks were proposed as an extension of RNNs to overcome these challenges. By introducing gated memory cells consisting of input, forget, and output gates, LSTMs can selectively retain or discard information over time. This mechanism enables LSTM networks to effectively learn both short-term variations and long-term dependencies in sequential data. As a result, LSTMs have been successfully applied to a wide range of time-series forecasting tasks, including speech recognition, financial forecasting, and weather prediction.

In this study, a deep learning–based approach using a stacked LSTM architecture is proposed for univariate time-series forecasting of daily maximum temperatures in Seattle. The model utilizes historical temperature data and a sliding window strategy to predict the temperature of the subsequent day. By stacking multiple LSTM layers, the network gains enhanced representational capacity, allowing it to capture complex temporal patterns present in the data. The primary objective of this research is to evaluate the effectiveness of stacked LSTM networks in modeling localized weather patterns and to demonstrate their potential as a reliable tool for short-term temperature forecasting.

II. Dataset Description

The experimental analysis in this study is conducted using the publicly available Seattle Weather Dataset, which contains daily meteorological observations recorded over multiple years. The dataset is widely used in time-series forecasting research due to its consistency, completeness, and representation of real-world weather variability. It includes several atmospheric parameters such as precipitation, wind speed, and temperature extremes, providing a comprehensive view of local weather conditions.

For the purpose of this research, the forecasting task is formulated as a univariate time-series problem. Among the available attributes, only the daily maximum temperature variable, denoted as `temp_max`, is utilized for model training and evaluation. This design choice allows the study to focus specifically on the temporal dynamics of temperature variation without introducing additional complexity from correlated meteorological features. By restricting the model to a single variable, the effectiveness of the proposed stacked LSTM architecture in capturing temporal dependencies can be evaluated more clearly.

The dataset consists of more than one thousand daily records, each corresponding to a unique calendar date. Prior to model development, the dataset is carefully examined to identify missing values, duplicate entries, and inconsistencies. Records with missing observations are handled appropriately to ensure data integrity, and the dataset is organized in chronological order to preserve temporal continuity. Maintaining the original temporal sequence is essential for time-series forecasting tasks, as shuffling the data would disrupt the inherent dependencies between consecutive observations.

The temporal coverage of the dataset enables the model to learn both short-term fluctuations and long-term seasonal trends in temperature data. Variations across different seasons, such as warmer summer periods and colder winter intervals, provide valuable contextual information for the learning process. This diversity in temperature patterns contributes to the robustness of the trained model and enhances its ability to generalize to unseen data.

By leveraging a real-world meteorological dataset and focusing on daily maximum temperature values, this study establishes a reliable foundation for evaluating deep learning–based time-series forecasting models under realistic conditions.

III. Data Preprocessing

Data preprocessing is a critical step in time-series forecasting, as the quality and structure of the input data directly influence the learning capability and performance of deep learning models. Meteorological datasets often contain noise, missing values, and variations in scale, which can adversely affect model convergence and predictive accuracy if not handled appropriately. In this study, a systematic preprocessing pipeline is designed to transform the raw Seattle Weather Dataset into a structured and normalized format suitable for training a stacked Long Short-Term Memory (LSTM) network.

The initial preprocessing step involves data inspection and cleaning. The dataset is examined for missing values, duplicate records, and inconsistencies in the temporal sequence. Ensuring the absence of missing observations is particularly important for recurrent neural networks, as gaps in time-series data can disrupt temporal continuity and lead to incorrect pattern learning. Any duplicate entries are removed, and the dataset is sorted chronologically to preserve the natural temporal order of observations. Maintaining sequential integrity is essential, as time-series models rely on historical dependencies between consecutive time steps.

Following data cleaning, the dataset is reduced to a univariate representation by selecting the daily maximum temperature attribute, denoted as `temp_max`. This simplification enables a focused analysis of temporal temperature patterns without introducing additional complexity from correlated meteorological variables such as precipitation or wind speed. By formulating the problem as a univariate forecasting task, the effectiveness of the proposed stacked LSTM architecture in learning sequential dependencies can be evaluated more clearly.

To convert the sequential time-series data into a supervised learning format, a sliding window technique is employed. A fixed window size of ten days is used, where temperature values from days $t-10$ to t constitute the input feature vector, and the temperature at day $t+1$ serves as the prediction target. This approach enables the model to learn short-term temporal relationships by observing recent historical trends while predicting future values. The sliding window is moved incrementally across the dataset, generating a large number of input–output pairs that collectively represent the temporal dynamics of the temperature series.

Once the input–output pairs are generated, the data is reshaped into a three-dimensional structure required by LSTM networks. Specifically, the input data is organized in the form $(samples, time\_steps, features)$, where each sample contains a sequence of ten consecutive temperature observations and a single feature dimension. This representation allows the LSTM layers to process temporal sequences effectively and retain relevant historical information through their internal memory states.

To ensure stable and efficient training, feature scaling is applied to the dataset. Temperature values are normalized using a scaling technique to bring them within a common numerical range. Normalization prevents features with larger magnitudes from dominating the learning process and improves gradient-based optimization during training. The scaling parameters are learned exclusively from the training data and then applied to the validation and test sets to avoid data leakage and ensure fair performance evaluation.

The dataset is then divided into training, validation, and testing subsets using a sequential split strategy. The first 800 days of data are allocated for training, the subsequent 200 days are reserved for validation, and the remaining observations are used for testing. This chronological partitioning closely simulates real-world forecasting scenarios, where future data

is unavailable during model training. The validation set is used to monitor model performance during training and to detect potential overfitting, while the test set provides an unbiased assessment of the model's generalization capability.

Through this comprehensive preprocessing pipeline, the raw meteorological data is transformed into a structured and learning-ready format. The combination of data cleaning, univariate selection, sliding window transformation, normalization, and sequential splitting ensures that the stacked LSTM model receives high-quality input data, thereby enhancing its ability to learn meaningful temporal patterns and produce reliable temperature forecasts.

IV. Model Architecture

The proposed forecasting framework is based on a deep recurrent neural network architecture utilizing stacked Long Short-Term Memory (LSTM) layers. LSTM networks represent an advanced variant of Recurrent Neural Networks (RNNs) that are specifically designed to model sequential data and address the shortcomings of traditional RNN architectures. In particular, conventional RNNs suffer from the vanishing and exploding gradient problems, which significantly limit their ability to learn long-term temporal dependencies. LSTM networks overcome these limitations through gated memory cells that regulate the flow of information over time, enabling the network to selectively retain relevant historical context while discarding redundant or noisy information.

The model architecture employed in this study follows a stacked LSTM design, where multiple LSTM layers are arranged sequentially to enhance the representational power of the network. Stacking LSTM layers allows the model to learn hierarchical temporal features, with lower layers capturing short-term patterns and higher layers modeling more abstract and long-range dependencies. This architectural choice is particularly beneficial for meteorological time-series data, which exhibit complex temporal structures such as daily fluctuations, seasonal variations, and long-term trends.

The input to the network consists of sequences generated using a sliding window approach with a fixed window size of ten days. Each input sample is represented as a three-dimensional tensor of shape $(samples, time\_steps, features)$, where the time_steps dimension corresponds to ten consecutive observations and the feature dimension is equal to one, reflecting the univariate nature of the forecasting task. This structured input representation ensures compatibility with LSTM layers and allows the model to process temporal sequences efficiently.

The network architecture is composed of four LSTM layers, each containing 50 hidden units. The first three LSTM layers are configured to return the full output sequence by enabling the `return_sequences=True` parameter. This configuration ensures that temporal information is preserved and propagated through successive layers, allowing deeper layers to access intermediate sequential representations. By maintaining the sequence output across layers, the model is able to capture temporal dependencies at multiple levels of abstraction.

The final LSTM layer is designed to output a single hidden state that summarizes the learned temporal context from the input sequence. This compressed representation encapsulates both recent and long-term information relevant to the forecasting task and serves as the input to the subsequent output layer. The transition from sequence outputs to a single contextual representation enables the model to generate a point prediction for the next time step.

To address the issue of overfitting, which is commonly encountered in deep learning models trained on relatively limited datasets, Dropout regularization is applied after each LSTM layer. A dropout rate of 0.2 is employed, meaning that 20% of the neurons are randomly deactivated during each training iteration. This stochastic regularization technique reduces the network's reliance on specific neurons, encourages redundancy in feature learning, and enhances the generalization capability of the model when applied to unseen data.

The output component of the architecture consists of a fully connected Dense layer with a single neuron. This layer performs a linear transformation of the final LSTM output to generate the predicted value of the daily maximum temperature. Since the forecasting task is formulated as a regression problem, no non-linear activation function is applied at the output layer. This design choice ensures that the model can produce continuous-valued predictions across the full range of temperature values.

The entire architecture is implemented using the TensorFlow and Keras deep learning frameworks, which provide a high-level interface for constructing, training, and evaluating neural network models. The modular design of the proposed architecture allows for straightforward experimentation with alternative configurations, such as varying the number of layers, hidden units, or regularization parameters.

Overall, the stacked LSTM architecture adopted in this study strikes a balance between model complexity and learning capacity. By leveraging multiple recurrent layers, gated memory mechanisms, and dropout regularization, the model is capable of effectively capturing the intricate temporal patterns present in historical weather data. This architectural design forms a robust foundation for accurate univariate time-series forecasting of daily maximum temperatures and demonstrates the suitability of deep recurrent networks for meteorological prediction tasks.

V. Training Configuration

The effectiveness of a deep learning model is not solely determined by its architecture but also heavily influenced by the training configuration and the selection of appropriate hyperparameters. In this study, careful consideration is given to the optimization strategy, loss function, data partitioning scheme, and training duration to ensure stable convergence and reliable forecasting performance. The training configuration is designed to simulate real-world forecasting conditions while minimizing overfitting and maximizing generalization capability.

The proposed stacked LSTM model is trained using the Adam (Adaptive Moment Estimation) optimizer, which is widely adopted in deep learning applications due to its computational efficiency and adaptive learning rate mechanism. Adam combines the advantages of both momentum-based optimization and adaptive gradient scaling, allowing it to adjust learning rates individually for each parameter. This property is particularly beneficial for training recurrent neural networks, where gradients can vary significantly across layers and time steps. The default Adam learning rate of 0.001 is employed, as it provides a balanced trade-off between convergence speed and training stability.

For the loss function, Mean Squared Error (MSE) is selected to quantify the discrepancy between predicted and actual temperature values. MSE is a standard choice for regression-based time-series forecasting tasks, as it penalizes larger prediction errors more severely and encourages the model to produce predictions that closely align with ground truth values. By minimizing the squared differences between predicted and observed temperatures, the model learns to reduce both systematic bias and high-variance fluctuations in its outputs.

The dataset is partitioned sequentially into training, validation, and testing subsets to preserve the temporal integrity of the time-series data. Unlike random splitting, which can introduce information leakage in sequential datasets, a chronological split ensures that the model is trained only on past observations and evaluated on future data. Specifically, the first 800 days of observations are allocated for training, the subsequent 200 days are used for validation, and the remaining samples constitute the test set. This partitioning strategy closely mirrors real-world forecasting scenarios, where future data is unavailable during model training.

The training process is conducted over 100 epochs, allowing the model sufficient opportunity to learn complex temporal patterns from the data. An epoch is defined as one complete pass through the training dataset. The choice of 100 epochs is empirically motivated to ensure convergence without excessive training that could lead to overfitting. During training, model performance is continuously monitored on the validation set to assess generalization behavior and detect potential divergence between training and validation losses.

A batch size of 32 is utilized during training, which represents a compromise between computational efficiency and gradient estimation accuracy. Smaller batch sizes introduce stochasticity that can help the optimizer escape local minima, while larger batch sizes provide more stable gradient updates. The selected batch size ensures efficient utilization of computational resources while maintaining sufficient variability in gradient updates to promote robust learning.

To further improve generalization and prevent overfitting, Dropout regularization is applied within the LSTM layers, as described in the model architecture section. Dropout is active only during training and is automatically disabled during inference, allowing the full network capacity to be utilized for prediction. This training-time regularization technique enhances the robustness of the learned representations and reduces sensitivity to noise in the training data.

The training procedure is implemented using the Keras high-level API within the TensorFlow framework. Keras provides built-in support for monitoring training and validation loss, enabling visual inspection of convergence behavior through loss curves. These curves are instrumental in diagnosing underfitting or overfitting and in validating the effectiveness of the chosen hyperparameters.

Overall, the training configuration adopted in this study is designed to ensure stable optimization, efficient convergence, and strong generalization performance. By combining adaptive optimization, appropriate loss selection, temporal data partitioning, and controlled training duration, the model is trained under conditions that closely resemble real-world deployment scenarios. This rigorous training setup plays a critical role in enabling the stacked LSTM network to accurately forecast daily maximum temperatures from historical weather data.

VI. Results and Evaluation

The evaluation of the proposed stacked Long Short-Term Memory (LSTM) network focuses on its convergence behavior, quantitative predictive accuracy, and generalization capability. The experimental results confirm that the model successfully captures temporal dependencies in univariate weather data, despite the inherent volatility of meteorological parameters. The training process, executed over 100 epochs, revealed a distinct three-phase learning progression that demonstrates the effectiveness of the optimization strategy. Initially, the model exhibited a rapid phase of feature extraction. Starting with a high error state of 196.32 MSE, the gradients were optimized significantly within the first 20 epochs, causing the training loss to plummet to 52.98. This sharp reduction indicates that the network quickly identified the dominant seasonal trends and primary periodicities in the data.

Following this initial descent, the learning rate of improvement decelerated, marking a transition to a stabilization phase. Between epochs 20 and 60, the loss curves flattened significantly, forming a distinct "elbow" as the model began to optimize for subtler temporal dependencies rather than coarse patterns. By epoch 60, the training loss had reached 10.11, demonstrating the network's capacity to retain long-term dependencies over the 10-day input window. In the final 40 epochs, the model achieved a stable state of convergence. The training loss settled around 8.88, while the validation loss stabilized near 9.80. Although minor oscillations were observed—characteristic of stochastic gradient descent with a batch size of 32—these fluctuations remained within a tight bound, confirming that the optimization had reached a robust minimum.

To strictly quantify the predictive performance, we utilized the Mean Squared Error (MSE) and derived the Root Mean Squared Error (RMSE) to provide a physically interpretable metric. The final model recorded a Training MSE of 8.88 and a Validation MSE of 9.80. The derived RMSE of approximately 3.13°C indicates that, on average, the model's forecasts for the next day's maximum temperature deviate from the actual

observed values by roughly 3 degrees. Given the high variance in the Seattle weather dataset, which is influenced by stochastic factors such as precipitation and wind speed, this error margin represents a robust baseline for a univariate model relying solely on historical temperature data.

A critical aspect of this study was ensuring that the deep learning model generalizes well to unseen data rather than memorizing the training set. The efficacy of the Dropout regularization (rate = 0.2), applied after each LSTM layer, is evidenced by the close correlation between the training and validation loss curves. Throughout the training duration, the validation loss tracked the training loss without significant divergence. At the termination point, the gap between training and validation MSE was negligible (< 1.0), confirming that the model learned robust, transferable features applicable to the validation period.

Finally, the experimental results validate the architectural decision to employ four stacked LSTM layers with 50 units each. This depth provided the necessary computational capacity to model the non-linear complexities of the weather time series. The ability to achieve a sub-10.0 MSE on the validation set confirms that the 10-day look-back window provided sufficient historical context for the LSTM cells to infer future states effectively. While the current performance is promising, the plateauing of the loss curve suggests that further gains would likely require the integration of multivariate features, such as pressure and humidity, rather than increased model complexity.

VII. Conclusion

This study successfully demonstrated the efficacy of a stacked Long Short-Term Memory (LSTM) neural network for the task of short-term weather forecasting, specifically predicting daily maximum temperatures using univariate time-series data. By leveraging a deep architecture composed of four stacked LSTM layers with 50 units each, the model proved capable of capturing the complex, non-linear relationships inherent in meteorological data. The network successfully modeled both rapid, high-frequency fluctuations and broader, long-term seasonal trends, validating the use of deep sequential learning for weather prediction tasks where historical context is paramount. The rigorous training configuration, which utilized the Adam optimizer, Mean Squared Error (MSE) loss function, and a chronological data partitioning strategy, ensured stable convergence and reliable performance metrics.

The experimental evaluation yielded a final Root Mean Squared Error (RMSE) of approximately 3.13°C on the validation set. Given the stochastic nature of weather patterns—which are influenced by variable factors such as precipitation, wind speed, and atmospheric pressure—an average deviation of roughly 3 degrees represents a robust baseline for a model that relies exclusively on past temperature records. Furthermore, the analysis of the training dynamics highlighted the critical role of regularization in deep learning applications. The application of Dropout regularization (rate = 0.2) effectively prevented overfitting, a common pitfall in sequential modeling. This was evidenced by the consistent alignment between the training and validation loss curves throughout the 100-epoch cycle, confirming that the model developed a generalized understanding of temporal weather patterns rather than simply memorizing the specific sequences within the training dataset.

The successful deployment of this architecture validates the hypothesis that deep LSTM networks can extract meaningful predictive features from limited historical windows, provided that the network depth and hyperparameters are appropriately tuned. However, the asymptotic behavior of the loss curve observed in the final epochs suggests that the predictive capacity of a purely univariate model may be approaching its theoretical limit. While the current results are promising, they also indicate a clear pathway for future research. Subsequent studies should focus on expanding the input feature space to include multivariate data. integrating auxiliary meteorological variables such as humidity, wind velocity, and barometric pressure would likely provide the additional context needed to resolve the residual errors observed in this study. Additionally, exploring advanced mechanisms such as attention layers could allow the model to better weigh the importance of specific past events, potentially enhancing its sensitivity to sudden or extreme weather shifts and further improving forecast accuracy.

VIII. Future Work

While the proposed stacked Long Short-Term Memory (LSTM) model has demonstrated robust performance in forecasting daily maximum temperatures, several avenues for further research and optimization remain. The primary limitation of the current approach is its univariate nature, relying solely on historical temperature data to predict future states. Weather systems are inherently complex and dynamic, governed by the interaction of multiple meteorological variables. Therefore, the most significant opportunity for enhancement lies in the transition to a multivariate forecasting framework. Future iterations of this model will integrate auxiliary features available in the dataset, such as precipitation levels, wind speed, and atmospheric pressure. Incorporating these variables would allow the network to learn the physical correlations between pressure systems, moisture, and temperature fluctuations, likely resolving the residual errors observed during sudden weather shifts where temperature history alone is an insufficient predictor.

Beyond data expansion, there is considerable scope for architectural innovation. While LSTMs are effective at mitigating the vanishing gradient problem, they can still struggle with extremely long-term dependencies or identifying which specific past time steps are most relevant to the current prediction. To address this, future work will explore the integration of Attention mechanisms. By adding an Attention layer to the existing LSTM stack, the model could dynamically assign weights to different time steps within the input window, allowing it to focus more heavily on critical past events—such as a sudden temperature drop three days prior—rather than treating

the entire 10-day window with uniform importance. Furthermore, evaluating Transformer-based architectures, which rely entirely on self-attention, could offer performance gains in computational efficiency and the ability to capture longer-range temporal relationships compared to recurrent networks.

Another critical area for future development is the extension of the forecasting horizon. The current model is designed for one-step-ahead prediction (forecasting the next day's temperature). However, practical meteorological applications often require multi-step forecasts covering a range of three to seven days. Future research will involve adapting the model to a Sequence-to-Sequence (Seq2Seq) architecture, enabling it to output a vector of future temperatures rather than a single scalar value. This would necessitate a rigorous re-evaluation of the loss function to ensure that errors do not accumulate and propagate over the extended forecast horizon.

Finally, to ensure the robustness and generalizability of the proposed solution, the model should be subjected to spatial cross-validation. Testing the architecture on weather datasets from geographically diverse regions—ranging from arid climates to tropical zones—will help determine if the learned hyperparameters are specific to the Seattle climate or if the underlying architectural strengths are transferrable. Concurrently, implementing automated hyperparameter optimization techniques, such as Bayesian Optimization or Genetic Algorithms, could refine the network configuration (e.g., number of layers, nodes, and dropout rates) more efficiently than manual tuning, potentially squeezing out the final percentage points of accuracy and minimizing the Mean Squared Error further.

REFERENCES

[1] *Neural Computation*, **S. Hochreiter and J. Schmidhuber**
1997
Topic: Model Architecture – Foundational paper describing the Long Short-Term Memory (LSTM) units used in this study.
https://doi.org/10.1162/neco.1997.9.8.1735

[2] *Proc. Int. Conf. Learn. Representations (ICLR)*, **D. P. Kingma and J. Ba**
2015
Topic: Optimization – Describes the Adam optimizer used to minimize the Mean Squared Error during training.
http://arxiv.org/abs/1412.6980

[3] *J. Mach. Learn. Res.*, **N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov**
2014
Topic: Regularization – Detailed explanation of the Dropout technique applied to prevent overfitting in the LSTM layers.
http://jmlr.org/papers/v15/srivastava14a.html

[4] *Software Documentation*, **F. Chollet et al.**
2015
Topic: Framework – The high-level Deep Learning API used to implement the stacked LSTM model.
https://keras.io

[5] *Proc. 12th USENIX Symp. Oper. Syst. Des. Implem. (OSDI)*, **M. Abadi et al.**
2016
Topic: Backend System – The large-scale machine learning system that serves as the backend for the Keras implementation.
https://www.tensorflow.org/about/bib

[6] *GitHub Repository*, **Vega Datasets**
2012
Topic: Dataset – Source of the historical Seattle weather data (rain, temp, wind) used for training and evaluation.
https://github.com/vega/vega/blob/main/docs/data/seattle-weather.csv

[7] *Advances in Neural Inf. Process. Syst.*, **A. Vaswani et al.**
2017
Topic: Future Work – Introduces the Transformer architecture and Attention mechanisms proposed for future model enhancements.
https://arxiv.org/abs/1706.03762

[8] *Advances in Neural Inf. Process. Syst.*, **I. Sutskever, O. Vinyals, and Q. V. Le**
2014
Topic: Future Work – Foundation for Sequence-to-Sequence learning, relevant for extending the model to multi-day forecasting.
https://arxiv.org/abs/1409.3215